

Inter-process Communication

Producer-Consumer Problem



By:

Dr. H. K. Azad

Assistant Professor

SCOPE

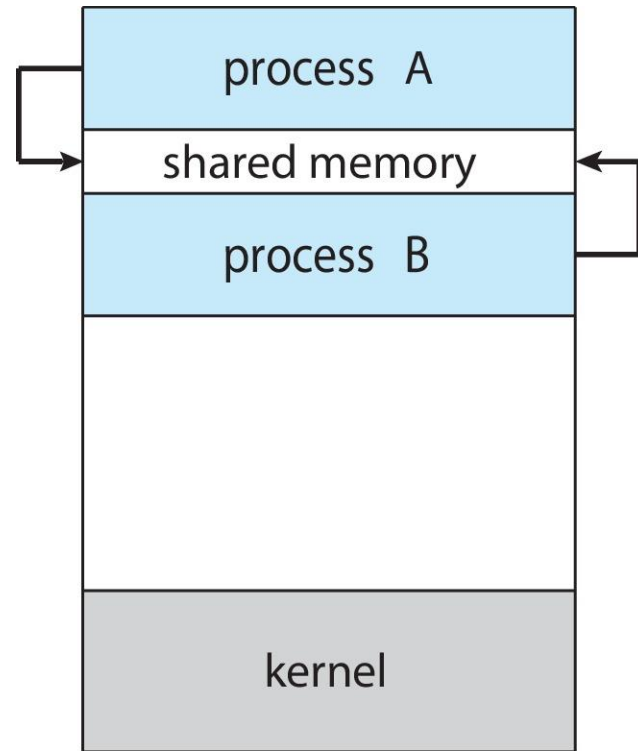
Email: hiteshwarkumar.azad@vit.ac.in

Interprocess Communication

- Processes within a system may be **independent** or **cooperating**
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
 - Convenience
- Cooperating processes need **interprocess communication (IPC)**
- Two models of IPC
 - **Shared memory** (under the control of users)
 - **Message passing** (under the control of OS)

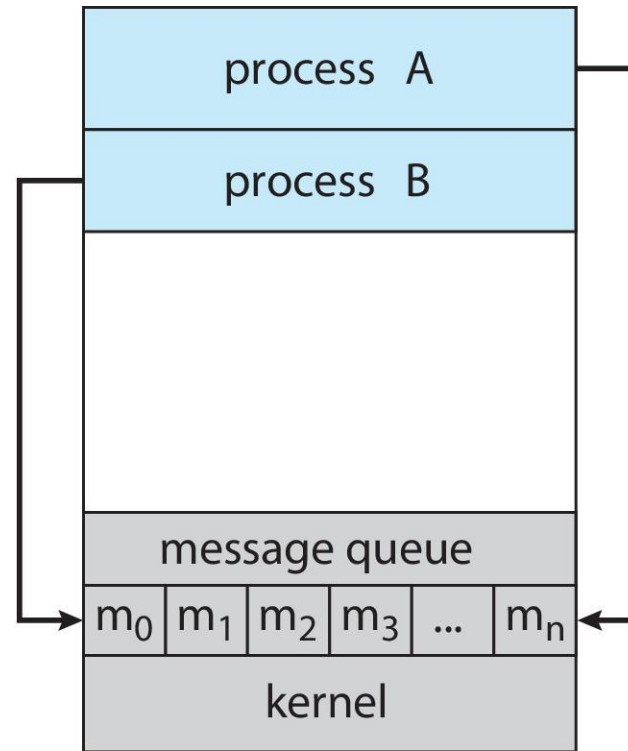
Communications Models

(a) Shared memory.



(a)

(b) Message passing.



(b)

Producer-Consumer Problem

- The Producer-Consumer problem is a classical multi-process synchronization problem, that is we are trying to achieve synchronization between more than one process.
- There is one Producer in the producer-consumer problem, Producer is producing some items, whereas there is one Consumer that is consuming the items produced by the Producer. The same memory buffer is shared by both producers and consumers which is of fixed-size.
- The problem is to make sure that the producer won't try to add data into the buffer if it's full and that the consumer won't try to remove data from an empty buffer.

Solution for the Producer

- Producer either go to sleep or discard data if the buffer is full.
- The next time the consumer removes an item from the buffer, and it notifies the producer who starts to fill the buffer again.

Solution for the Consumer

- Consumer can go to sleep if it finds the buffer to be empty.
- The next time the producer puts data into the buffer, it wakes up the sleeping consumer.

Producer-Consumer Problem

- Paradigm for cooperating processes:
 - ***producer*** process produces information that is consumed by a ***consumer*** process
- Two variations:
 - **unbounded-buffer** places no practical limit on the size of the buffer:
 - Producer never waits
 - Consumer waits if there is no buffer to consume
 - **bounded-buffer** assumes that there is a fixed buffer size
 - Producer must wait if all buffers are full
 - Consumer waits if there is no buffer to consume

Shared Memory Solution

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.

Bounded-Buffer – Shared-Memory Solution

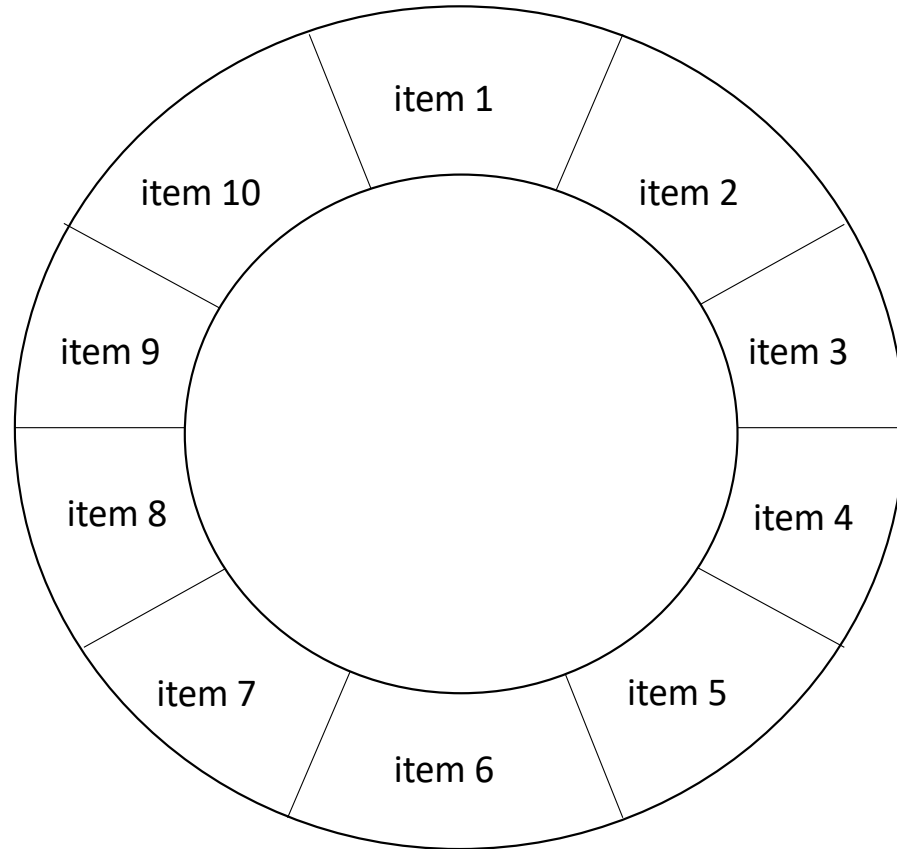
- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Solution presented in next slides is correct, but can only use **`BUFFER_SIZE-1`** items; that is: 9 items

Bounded-Buffer (Cont.)



Producer Code

```
int count = 0;
void producer( void )
{
    int itemP;
    while(1)
    {
        Produce_item(item P)
        while( count == n);
        buffer[ in ] = item P;
        in = (in + 1)mod n
        count = count + 1;
    }
}
```

Load Rp, m[count]
Increment Rp
Store m[count], Rp

Buffer[in] = itemP $\rightarrow$ Buffer[5] = F. (F is inserted now)
 $in = (in + 1) \bmod n \rightarrow (5 + 1) \bmod 8 \rightarrow 6$, therefore in = 6;
(next empty buffer)

After insertion of F, Buffer looks like this
Where **out = 0**, but **in = 6**

0	A
1	B
2	C
3	D
4	E
5	F
6	
7	

Buffer

(pointer to the next empty space) $\rightarrow$ in = 5
(pointer to the first filled space) $\rightarrow$ out = 0
(number of elements in Buffer) $\rightarrow$ count = 5
(Size of Buffer) $\rightarrow$ n = 8

0	A
1	B
2	C
3	D
4	E
5	
6	
7	

Consumer Code

```
int count;  
void consumer(void)  
{  
    int itemC;  
    while(1)  
    {  
        while( count == 0 );  
        itemC = buffer[ out ];  
        out = ( out + 1 ) mod n;  
        count = count - 1;  
    }  
}
```



Load Rc, m[count]
Decrement Rc
Store m[count], Rc

itemC = Buffer[out] → itemC = A (since out is 0)
out = (out + 1) mod n → (0 + 1) mod 8 → 1, therefore
out = 1 (first filled position)

A is removed now

After removal of A, Buffer look like this

Where **out = 1**, and **in = 6**

0	
1	B
2	C
3	D
4	E
5	F
6	
7	

Producer Process – Shared Memory

```
item next_produced;

while (true) {
    /* produce an item in next produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

Consumer Process – Shared Memory

```
item next_consumed;

while (true) {
    while (in == out)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next consumed */
}
```

What about Filling all the Buffers?

- Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers.
- We can do so by having an integer `counter` that keeps track of the number of full buffers.
- Initially, `counter` is set to 0.
- The integer `counter` is incremented by the producer after it produces a new buffer.
- The integer `counter` is and is decremented by the consumer after it consumes a buffer.

Producer

```
while (true) {  
    /* produce an item in next produced */  
  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```


Consumer

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in next consumed */  
}
```

IPC – Message Passing

- Processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
 - `send(message)`
 - `receive(message)`
- The *message* size is either fixed or variable

Message Passing (Cont.)

- If processes P and Q wish to communicate, they need to:
 - Establish a [communication link](#) between them
 - Exchange messages via send/receive
- Implementation issues:
 - How are links established?
 - Can a link be associated with more than two processes?
 - How many links can there be between every pair of communicating processes?
 - What is the capacity of a link?
 - Is the size of a message that the link can accommodate fixed or variable?
 - Is a link unidirectional or bi-directional?

Implementation of Communication Link

- Physical:
 - Shared memory
 - Hardware bus
 - Network
- Logical:
 - Direct or indirect
 - Synchronous or asynchronous
 - Automatic or explicit buffering

Direct Communication

- Processes must name each other explicitly:
 - **send** (*P*, *message*) – send a message to process P
 - **receive**(*Q*, *message*) – receive a message from process Q
- Properties of communication link
 - Links are established automatically
 - A link is associated with exactly one pair of communicating processes
 - Between each pair there exists exactly one link
 - The link may be unidirectional, but is usually bi-directional

Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
 - Each mailbox has a unique id
 - Processes can communicate only if they share a mailbox
- Properties of communication link
 - Link established only if processes share a common mailbox
 - A link may be associated with many processes
 - Each pair of processes may share several communication links
 - Link may be unidirectional or bi-directional

Indirect Communication (Cont.)

- Operations
 - Create a new mailbox (port)
 - Send and receive messages through mailbox
 - Delete a mailbox
- Primitives are defined as:
 - **Send**(*A, message*) – send a message to mailbox A
 - **receive**(*A, message*) – receive a message from mailbox A

Indirect Communication (Cont.)

- Mailbox sharing
 - P_1 , P_2 , and P_3 share mailbox A
 - P_1 sends; P_2 and P_3 receive
 - Who gets the message?
- Solutions
 - Allow a link to be associated with at most two processes
 - Allow only one process at a time to execute a receive operation
 - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

Thanks